

Cloud Platforms AWS / GCP / Azure

Data Engineering in the Cloud

■■ CLOUD

■ LEARNING OUTCOME

By the end of this module you will navigate AWS, GCP, and Azure data services, compare managed offerings, use boto3 and GCP SDK to build cloud pipelines, and understand cost optimisation for data workloads.

■ Skills You Will Gain

- Compare S3, GCS, and Azure Blob for data lake storage
- Understand serverless vs managed compute for data processing
- Use AWS boto3 SDK to read/write S3 and trigger Glue jobs
- Use Google BigQuery Python SDK for querying and loading
- Compare Redshift, BigQuery, and Synapse Analytics
- Apply cloud cost optimisation patterns for DE workloads

■ Cloud DEs earn 40% more than on-premise DEs. AWS, GCP, and Azure certifications each add ■3–8 LPA. Every enterprise and startup has moved to cloud — this is where the jobs are.

SECTION 1

Cloud DE Service Comparison

Service Category	AWS	GCP	Azure
Object Storage	S3	Cloud Storage (GCS)	Azure Blob Storage / ADLS Gen2
Data Warehouse	Redshift	BigQuery	Synapse Analytics
Managed Spark	EMR	Dataproc	HDInsight
Serverless ETL	AWS Glue	Dataflow / Dataproc	Azure Data Factory
Stream Processing	Kinesis	Pub/Sub + Dataflow	Event Hubs + Stream Analytics
Orchestration	MWAA (Airflow)	Cloud Composer	ADF Pipelines / Fabric
NoSQL	DynamoDB	Firestore / Bigtable	Cosmos DB
ML Platform	SageMaker	Vertex AI	Azure Machine Learning
Data Catalog	AWS Glue Catalog	Dataplex	Microsoft Purview

SECTION 2

AWS for Data Engineers

```
import boto3 import pandas as pd from io import BytesIO # S3 – object storage operations s3 = boto3.client('s3', region_name='us-east-1') # Upload DataFrame as Parquet to S3 data lake buffer = BytesIO() df.to_parquet(buffer, index=False, engine='pyarrow') s3.put_object(Bucket='my-data-lake', Key='sales/year=2024/month=01/day=15/orders.parquet', Body=buffer.getvalue()) # Read Parquet from S3 obj = s3.get_object(Bucket='my-data-lake', Key='sales/year=2024/month=01/day=15/orders.parquet') df = pd.read_parquet(BytesIO(obj['Body'].read())) # List files in S3 prefix paginator = s3.get_paginator('list_objects_v2') for page in paginator.paginate(Bucket='my-data-lake', Prefix='sales/year=2024/'): for obj in page.get('Contents', []): print(obj['Key'], obj['Size']) # Trigger AWS Glue ETL job glue = boto3.client('glue', region_name='us-east-1') response = glue.start_job_run( JobName='sales-etl-pipeline', Arguments={'--date': '2024-01-15', '--env': 'prod'}) print(f'Glue Job Run ID: {response["JobRunId"]}')
```

SECTION 3

Google BigQuery

Why BigQuery is Different

Serverless: no infrastructure to provision, manage, or scale Columnar: only reads columns needed — aggregations over millions of rows in seconds Separation of storage and compute: pay per query, not per server Auto-scaling: handles 1 query or 1000 concurrent queries the same way Built-in ML: BigQuery ML — run ML models with SQL inside the warehouse

```
from google.cloud import bigquery import pandas as pd client = bigquery.Client(project='my-gcp-project') # Run SQL query and get results as DataFrame query = '''SELECT region, DATE_TRUNC(order_date, MONTH) AS month, SUM(revenue) AS total_revenue, COUNT(DISTINCT customer_id) AS unique_customers FROM `my-project.sales.fact_orders` WHERE order_date >= '2024-01-01' GROUP BY 1, 2 ORDER BY month, total_revenue DESC ''' df =
```

```
client.query(query).to_dataframe() # Load DataFrame to BigQuery (overwrite table) job_config =
bigquery.LoadJobConfig( write_disposition='WRITE_TRUNCATE', # overwrite existing data
autodetect=True # auto-detect schema ) load_job = client.load_table_from_dataframe( df,
'my-project.sales.monthly_summary', job_config=job_config ) load_job.result() # wait for completion
print(f'Loaded {load_job.output_rows} rows')
```

SECTION 4

Cloud Cost Optimisation for DE

Pattern	How to Apply	Typical Savings
Use Parquet not CSV	All S3/GCS data in Parquet or Delta format	60-80% storage, 3-10x query speed
Partition S3 data	Write with partitionBy('year','month','day')	90%+ scan reduction with partition pruning
Right-size clusters	Spot/Preemptible instances for Spark batch jobs	50-70% compute cost reduction
Query only what you need	SELECT specific columns, not SELECT *	80%+ BigQuery scan cost reduction
Use Reserved capacity	Purchase Snowflake/Redshift reserved capacity up to 70% discount vs on-demand	30-60%
Lifecycle policies	S3: move data to Glacier after 90 days	80%+ storage cost reduction for old data
Cluster auto-scaling	EMR/Dataproc autoscaling — scale down when idle	40%+ compute savings

■ PRACTICE Q & A

Q1. What is the main advantage of BigQuery over traditional databases?

A: *Serverless — no infrastructure to manage. Scales automatically. Columnar storage enables petabyte-scale queries in seconds. Pay per query (bytes scanned). No indexes needed — full table scans are fast.*

Q2. What is AWS S3's role in a data lake architecture?

A: *Cheap, durable (11 nines), infinitely scalable object storage. Stores raw data in any format (JSON, CSV, Parquet). Foundation of the data lake — source of truth before transformation. Integrates with all AWS services.*

Q3. What is AWS Glue and how does it help data engineers?

A: *Serverless ETL service. Glue Crawler auto-discovers schema from S3 and builds a Data Catalog. Glue Jobs run PySpark ETL code without managing servers. Glue Catalog integrates with Athena, Redshift, EMR.*

Q4. How does partitioning S3 data save money in BigQuery/Athena?

A: *When you partition by date (s3://bucket/data/year=2024/month=01/), queries with WHERE date='2024-01' only scan that partition. S3/BigQuery skip all other partitions — reads 100x less data = 100x cheaper queries.*

Q5. What is the difference between Redshift and BigQuery?

A: *Redshift: AWS MPP columnar database, provisioned clusters (or Serverless), best for complex SQL and existing AWS workloads. BigQuery: GCP fully serverless, auto-scales, pay per query — no cluster management ever.*

■ ■ Cloud DE Cheat Sheet	
boto3.client('s3')	Connect to AWS S3
s3.put_object(Bucket, Key, Body)	Upload to S3
s3.get_object(Bucket, Key)	Download from S3
bigquery.Client(project)	Connect to BigQuery
client.query(sql).to_dataframe()	BQ query → DataFrame
WRITE_TRUNCATE	Overwrite BQ table
WRITE_APPEND	Append to BQ table
glue.start_job_run(JobName)	Trigger AWS Glue job

<code>partitionBy('year','month')</code>	Partition Parquet on write
EMR / Dataproc	Managed Spark cluster
Spot / Preemptible instances	50-70% cheaper compute
S3 Lifecycle → Glacier	Cheap archival after 90 days